实验四、SLAM 综合应用

一、 实验目的

1、了解并掌握 SLAM 激光雷达建图原理。
2、学会自己搭建 SLAM 激光雷达地图。
3、了解并掌握 ROS 机器人的自主导航与避障。

二、 实验原理及说明

1、 SLAM 激光雷达建图

SLAM (Simultaneous Localization and Mapping，即时定位与地图构建)最早由 Smith、Self 和 Cheeseman 于 1988 年提出。作为一种基础技术，SLAM 从最早的军事用途到今天的扫地机器人，吸引了一大批研究者和爱好者，同时也使这项技术逐步走入普通消费者的视野。

使用 ROS 实现机器人的 SLAM 和自主导航等功能是非常方便的，因为有较多现成的功能包可供开发者使用，如 gmapping、hector_slam、cartographer、rgbdslam、ORB_SLAM、move_base、amcl 等。

1.1 GMapping 建图算法理论概念

GMapping 的理论概念较为简单，它是基于 Rao-Blackwellized 粒子滤波(RBPF）算法实现的开源 SLAM 算法，GMapping 算法将定位与建图的过程分离，先通过粒子滤波算法做定位，并通过粒子与已产生地图进行 scanMatch，然后不断矫正里程计误差并添加新的 scan 做为地图。Gmapping 在 RBpf 算法上做了两个主要的改进:改进提议分布和选择性重采样。

同其他建图算法相比，GMapping 算法的优点与缺点也比较显著。

GMapping 算法的优点在于可以实时构建室内地图，在构建小场景地图所需的计算量较小且精度较高。相比 Hector SLAM 对激光雷达频率要求低、鲁棒性高(Hector 在机器人快速转向时很容易发生错误匹配，建出的地图发生错位，原因主要是优化算法容易陷入局部最小值)；而相比 Cartographer 在构建小场景地图时，Gmapping 不需要太多的粒子并且没有回环检测因此计算量小于 Cartographer 而精度并没有差太多。Gmapping 有效利用了车轮里程计信息，这也是 Gmapping 对激光雷达频率要求低的原因:里程计可以提供机器人的位姿先验。而 Hector 和 Cartographer 的设计初衷不是为了解决平面移动机器人定位和建图，Hector 主要用于救灾等地面不平坦的情况，因此无法使用里程计。而 Cartographer 是用于手持激光雷达完成 SLAM 过程，也就没有里程计可以用。

GMapping 算法的缺点是随着场景增大所需的粒子增加，因为每个粒子都携带一幅地图，因此在构建大地图时所需内存和计算量都会增加。因此不适合构建大场景地图。并且没有回环检测，因此在回环闭合时可能会造成地图错位，虽然增加粒子数目可以使地图闭合但是以增加计算量和内存为代价。所以不能像 Cartographer 那样构建大的地图，虽然论文生成几万平米的地图，但实际我们使用中建的地图没有几千平米时就会发生错误。Gmapping 和 Cartographer 一个是基于滤波框架 SLAM 另一个是基于优化框架的 SLAM，两种算法都涉及到时间复杂度和空间复杂度的权衡。Gmapping 牺牲空间复杂度保证时间复杂度，这就造成 Gmapping 不适合构建大场景地图，试想一下要构建 200×200 米的环境地图，栅格分辨率选择 5 厘米，每个栅格占用一字节内存，那么一个粒子携带的地图就需要 16M 内存，如果是 100 个粒子就需要 1.6G 内存。如果地图变成 500 乘 500 米，粒子数为 200 个，可能电脑就要崩溃了。翻看 Cartographer 算法，优化相当于地图中只用一个粒子，因此存储空间比

较 Gmapping 会小很多倍，但计算量大，一般的笔记本很难跑出来好的地图，甚至根本就跑不动。优化图需要复杂的矩阵运算，这也是谷歌为什么还要弄个 ceres 库出来的原因。

GMapping 的算法流程如下图所示:

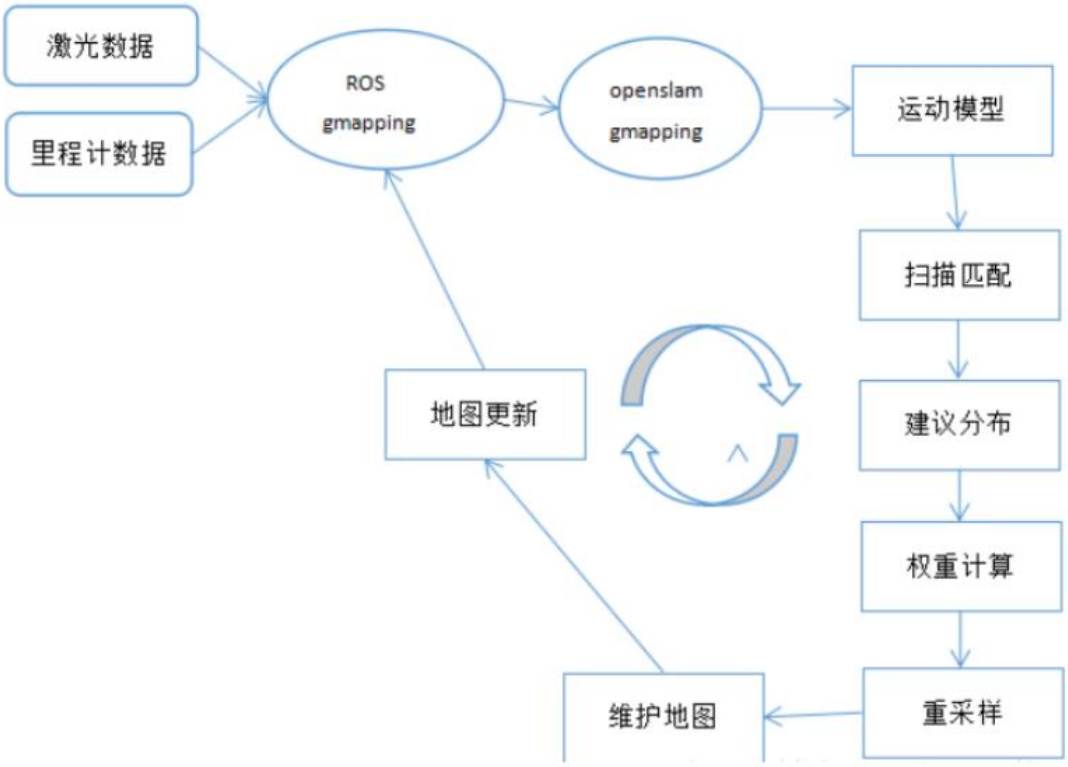


图 2.1 GMapping 算法流程

由图上可以看出，在 ROS 中的 slam_gmapping 软件包依赖于开源的 openslam_gmapping 算法包，也就是说，在 ROS 中的 gmapping 软件包是对 openslam_gmapping 算法包的 ROS 封装实现。

1.2 slam_gmapping 软件包的 ROS 框架

在 ROS 中，机器人使用 slam_gmapping 软件包实现了 GMapping 算法建图，在塔克机器人中，已通过二进制方法安装了 gmapping 软件包，用户无需手动安装。

slam_gmapping 节点所需要的 topic 数据及其发布的 topic 数据如下。

slam_gmapping 收听话题

·**tf**(tf/tfMessage)

gmapping 软件包通过查询 tf 变化，获取机器人基座标系与雷达的变换关系，以及获取机器人里程计的定位信息。

·**scan**(sensor_msgs/LaserScan)

用于创建地图的激光雷达数据

slam_gmapping 发布话题

·**map**(nav_msgs/OccupancyGrid)

slam_gmapping 参数

·**throttle_scans** 算法在每多少次的扫描中处理一次数据（将其设置为更高的数字以跳过更多扫描)。

·**base frame** 机器人基坐标系名称。

·**map_frame** 地图坐标系名称

·**odom_frame** 里程计坐标系名称

·**map_update_interval** 两次更新之间的时间间隔（以秒为单位)。降低此数字会更频繁地更新占用网格，以增加计算负荷为代价。

·**maxUrange** 激光的最大可用范围。激光数据被裁剪为该值。

·**maxRange** 传感器的最大范围。如果传感器范围内没有障碍的区域应显示为地图上的自由空间，则将该值设置为 maxUrange<实际传感器的最大范围<=maxRange。

·sigma 贪心端点匹配法使用的 sigma 值。

·kernelSize 寻找激光匹配关系的核大小。

·lstep 平移的优化步长。

·astep 旋转的优化步长。

·iterations scanmatcher 的迭代次数。

·lsigma 用于似然计算的激光的 sigma 值。

·ogain 评估似然时要使用的增益，以平滑重采样效果。

·**lskip** 每次扫描中要跳过的光束数。仅获取第(n + 1)条激光束以计算匹配项（O=捕获所有射线)

·**minimumScore** 考虑扫描匹配结果良好的最低分数。当使用有限范围(例如 5m)的激光扫描仪时，可以避免在大的空地中跳跃姿势的估计。分数上升到 600+，例如在遇到跳跃估计问题时尝试 50。

·srr 在线性方程中的里程计的线性误差。

·srt 在旋转方程中的里程计的线性误差。

·str 在线性方程中的里程计的旋转误差。

·stt 在旋转方程中的里程计的旋转误差。

·**linearUpdate** 机器人每次平移此距离时都要进行扫描更新。

·**angularUpdate** 机器人每次旋转这些弧度时都要进行扫描更新。

·temporalUpdate 如果最后处理的扫描早于更新时间(以秒为单位)，则处理扫描。小于零的值(-1)将关闭基于时间的更新。

·resampleThreshold 基于 Neff 的重采样阈值。

·**particles** 滤波器中的采样粒子数。

·xmin 最小初始地图尺寸(m)

·ymin 最小初始地图尺寸(m)

·xmax 最大初始地图尺寸(m)

·ymax 最大初始地图尺寸(m)

·delta 地图的分辨率(每个占用格块的长度 m)

·llsamplerange 平移采样范围的似然域。

·llsamplestep 平移采样步骤的似然域。

·transform_publish_period 转换发布之间的时间间隔（以秒为单位)。要禁用广播转换，请设置为 0。

·occ_thresh 映射的占用率阈值。占用率较高的单元被视为已占用(即在结果 sensor_msgs / LaserScan 中设置为 100)。

注:黑色加粗的参数为常用算法调节参数，用户可根据参数描述调整算法参数以达到不同的建图效果，其余参数为算法内部参数，除非对算法有深刻理解，否则不建议调整该参数。

1.3 在 ROS 中部署 GMapping 建图算法

在塔克机器人中，GMapping 建图算法的启动文件位于机器人端 ~/ros_ws/src/xtark_nav/launch/xtark_mappinq_qmappinq.launch。内容如下:

```
xtark@xtark-robot: ~/ros_ws/src/xtark_nav/launch
1 <launch>
2         <arg name="resolution" default="480p"/>
3         <!-- 启动底盘及激光雷达、手柄驱动包 -->
4         <include file="$(find xtark_driver)/launch/xtark_bringup.launch" />
5
6         <!-- 启动USB摄像头驱动包 -->
7         <include file="$(find xtark_driver)/launch/xtark_camera.launch">
8                 <arg name="resolution" value="$(arg resolution)"/>
9         </include>
10
11        <!-- 启动GMapping建图算法包 -->
12        <include file="$(find xtark_nav)/launch/include/gmapping_base.launch" />
13
14        <!-- 启动APP相关接口服务 -->
15        <include file="$(find rosbridge_server)/launch/rosbridge_websocket.launch"/>
16        <node name="robot_pose_publisher" pkg="robot_pose_publisher" type="robot_pose_publisher"/>
17        <arg name="debug" default="false"/>
18        <node pkg="world_canvas_server" type="world_canvas_server" name="world_canvas_server" args="$(a
  rg debug)">
19                <param name="start_map_manager" value="true"/>
20                <param name="auto_save_map" value="false"/>
21        </node>
22        <node pkg="world_canvas_server" type="map_manager.py" name="map_manager" />
23
24 </launch>
```

图 2.2 启动文件

其核心启动文件为:

第 4 行:启动了 xtark_bringup.launch 文件，该文件为塔克机器人底层 ROS 驱动包，提供了 GMapping 算法所需要的里程计(odom)消息。

第 12 行:启动了 gmapping_base.launch，该 launch 文件启动了核心的 GMapping 算法包及对 GMapping 算法的参数进行初始化。

gmapping_base.launch (~/ros_ws/src/xtark_nav/launch/include/)）文件内容如下;

```
<launch>
  <arg name="scan_topic"  default="scan" />
  <arg name="base_frame"  default="base_footprint"/>
  <arg name="odom_frame"  value="odom"/>

  <node pkg="gmapping" type="slam_gmapping" name="slam_gmapping" output="screen" respawn="true" >
    <param name="base_frame" value="$(arg base_frame)"/>
    <param name="odom_frame" value="$(arg odom_frame)"/>
    <!--param name="map_update_interval" value="1"/-->
    <param name="map_update_interval" value="0.4"/>
    <param name="maxUrange" value="8.0"/>
    <param name="maxRange" value="12.0"/>
    <param name="sigma" value="0.05"/>
    <!--param name="kernelSize" value="3"/-->
    <param name="kernelSize" value="1"/>
    <param name="lstep" value="0.05"/>
    <param name="astep" value="0.05"/>
    <param name="iterations" value="5"/>
    <param name="lsigma" value="0.075"/>
    <param name="ogain" value="3.0"/>
    <param name="lskip" value="0"/>
    <!--param name="minimumScore" value="30"/ -->
    <param name="minimumScore" value="100"/>
    <param name="srr" value="0.01"/>
    <param name="srt" value="0.02"/>
    <param name="str" value="0.01"/>
    <param name="stt" value="0.02"/>
    <param name="linearUpdate" value="0.05"/>
    <param name="angularUpdate" value="0.0436"/>
    <param name="temporalUpdate" value="-1.0"/>
    <param name="resampleThreshold" value="0.5"/>
    <param name="particles" value="8"/>
  <!--
    <param name="xmin" value="-50.0"/>
    <param name="ymin" value="-50.0"/>
    <param name="xmax" value="50.0"/>
    <param name="ymax" value="50.0"/>
  make the starting size small for the benefit of the Android client's memory...
  -->
    <param name="xmin" value="-50.0"/>
    <param name="ymin" value="-50.0"/>
    <param name="xmax" value="50.0"/>
    <param name="ymax" value="50.0"/>

    <param name="delta" value="0.05"/>
    <param name="llsamplerange" value="0.01"/>
    <param name="llsamplestep" value="0.01"/>
    <param name="lasamplerange" value="0.005"/>
    <param name="lasamplestep" value="0.005"/>
    <remap from="scan" to="$(arg scan_topic)"/>
  </node>
</launch>
```

图 2.3 文件内容

参数的详细解析，参照上一章节，用户可自行调整算法参数，以达到自己想要的效果。

2、 SLAM 导航理论

导航的关键是机器人定位和路径规划两大部分。针对这两个核心，ROS 提供了以下两个功能包。

1) move_base：实现机器人导航中的最优路径规划。

2) amcl：实现二维地图中的机器人定位。

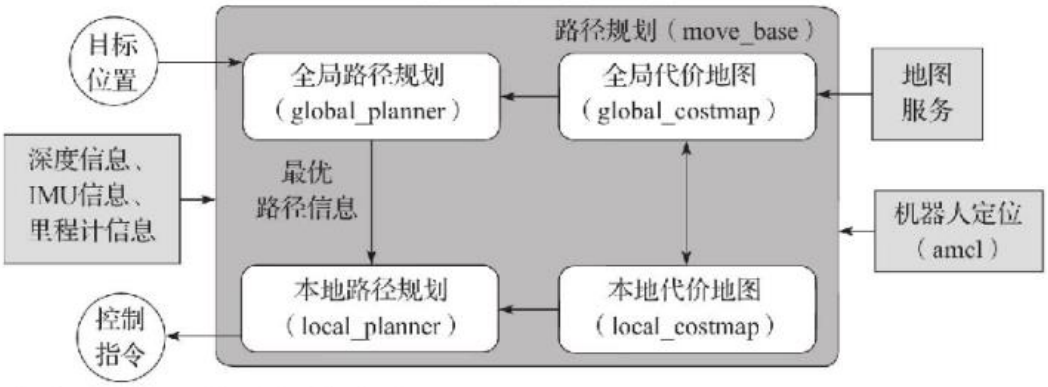


图 2.4 导航框架

机器人只需要发布必要的传感器信息和导航的目标位置，ROS 即可完成导航功能。在该框架中，move_base 功能包提供导航的主要运行、交互接口。为了保障导航路径的准确性，机器人还要对自己所处的位置进行精确定位，这部分功能由 amcl 功能包实现。

首先，导航功能包需要采集机器人的传感器信息，以达到实时避障的效果。这就要求机器人通过 ROS 发布 sensor_msgs/LaserScan 或者 sensor_msgs/PointCloud 格式的消息，也就是二维激光信息或者三维点云信息。

其次，导航功能包要求机器人发布 nav_msgs/Odometry 格式的里程计信息，同时也要发布相应的 TF 变换。

最后，导航功能包的输出是 geometry_msgs/Twist 格式的控制指令，这就要求机器人控制节点具备解析控制指令中线速度、角速度的能力，并且最终通过这些指令控制机器人完成相应的运动。

move_base 是 ROS 中完成路径规划的功能包，主要由以下两大规划器组成。

全局路径规划(global_planner)，全局路径规划是根据给定的目标位置和全局地图进行总体路径的规划。在导航中，使用 Dijkstra 或 A*算法进行全局路径的规划，计算出机器人到目标位置的最优路线，作为机器人的全局路线。

本地实时规划(local_planner)，在实际情况中，机器人往往无法严格按照全局路线行驶，所以需要针对地图信息和机器人附近随时可能出现的障碍物规划机器人每个周期内应该行驶的路线，使之尽量符合全局最优路径。

三、 实验内容及步骤

1、 GMapping 建图算法实践操作

通过 SSH 命令连接到机器人，运行 xtark_nav 导航驱动包的 xtark_slam 建图节点。

```
xtark@xtark-robot:~$ roslaunch xtark_nav xtark_slam.launch
```

说明: xtark_slam 建图节点默认为 gmapping 建图算法，也可以通过 slam_methods 参数配置不同的建图算法，目前可配置“gmapping”"hector""karto" "cartographer""frontier"五种建图算法。

此时节点正在启动，终端会输出启动信息，等待出现下图启动完成界面。

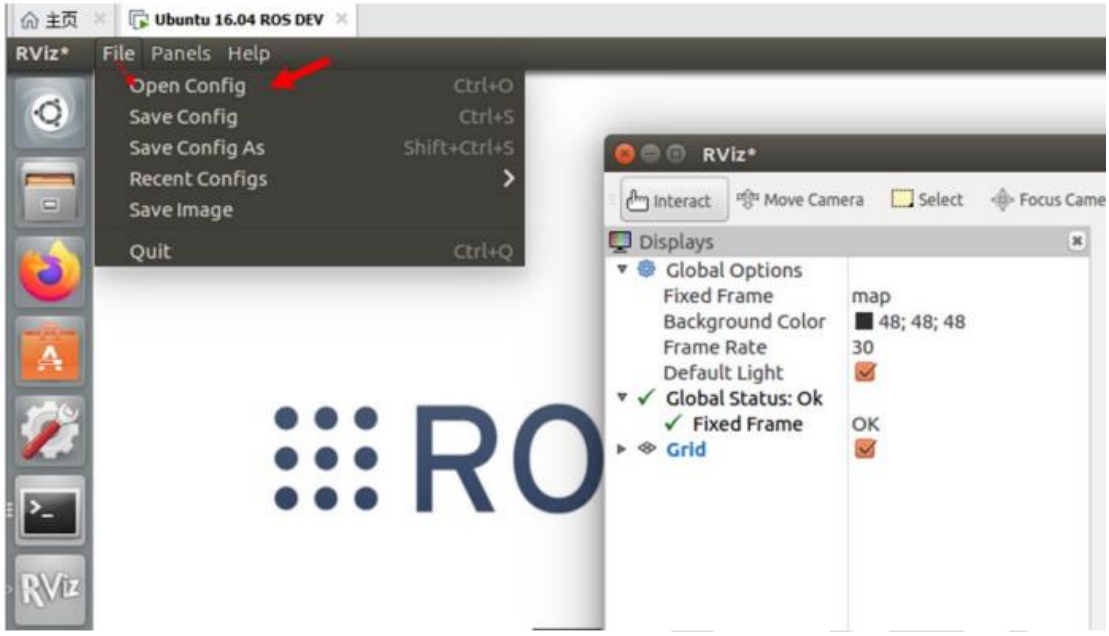
```
[ INFO] [1574066591.157231583]: Firmware Ver: 1.25
[ INFO] [1574066591.157689557]: Hardware Rev: 5
[ INFO] [1574066591.159745577]: RPLidar health status : 0
[ INFO] [1574066591.712107993]: current scan mode: Express, max_distance: 12.0 m
, Point number: 4.0K , angle_compensate: 1
[ INFO] [1574066592.350363815]: Laser is mounted upwards.
 -maxUrange 8 -maxUrange 12 -sigma     0.05 -kernelSize 1 -lstep 0.05 -lobsGain
3 -astep 0.05
 -srr 0.01 -srt 0.02 -str 0.01 -stt 0.02
 -linearUpdate 0.05 -angularUpdate 0.0436 -resampleThreshold 0.5
 -xmin -50 -xmax 50 -ymin -50 -ymax 50 -delta 0.05 -particles 8
[ INFO] [1574066592.356543579]: Initialization complete
update frame 0
update ld=0 ad=0
Laser Pose= 0 0 -3.13286
m_count 0
Registering First Scan
```

再新建一个终端，在虚拟机端运行 RVIZ 可视化工具。

```
xtark@xtark-vmpc:~$ rosrun rviz rviz
```

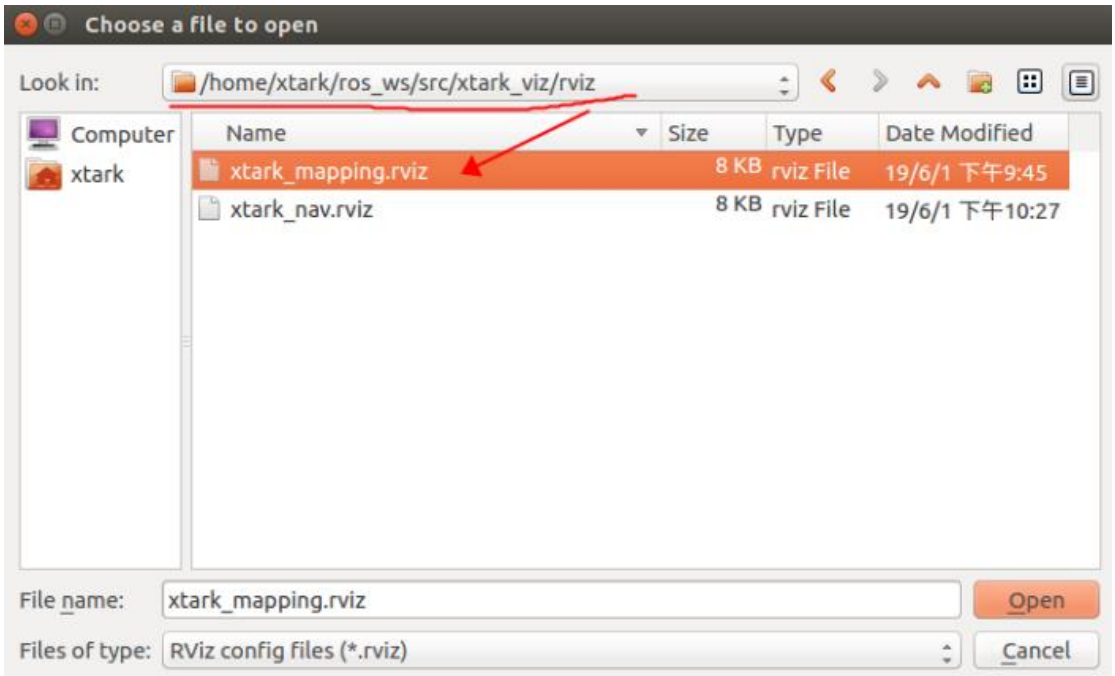
RVIZ 的配置项较多，为了方便使用，我们已做好适合建图和导航的配置文件，用户打开对应配置文件即可。更多 RVIZ 配置项资料，请用户自行查阅资料。

选择“File”菜单下的“Open Config”选项。温馨提示，File 菜单位置不同于 Windows，不在软件窗口上，在整个界面的左上角，不了解 ubuntu 的用户可能会找不到。

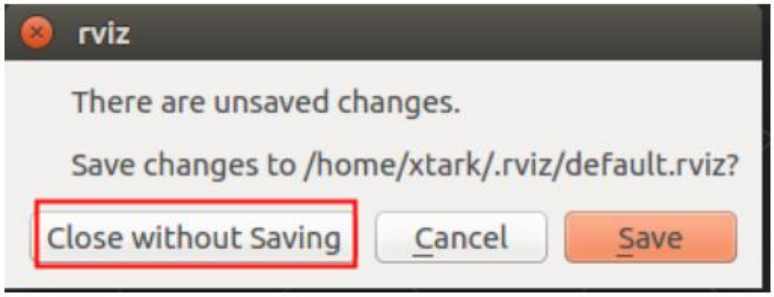


打开已经设置好的建图配置文件“xtark_mapping.rviz”，文件路径参考下图。

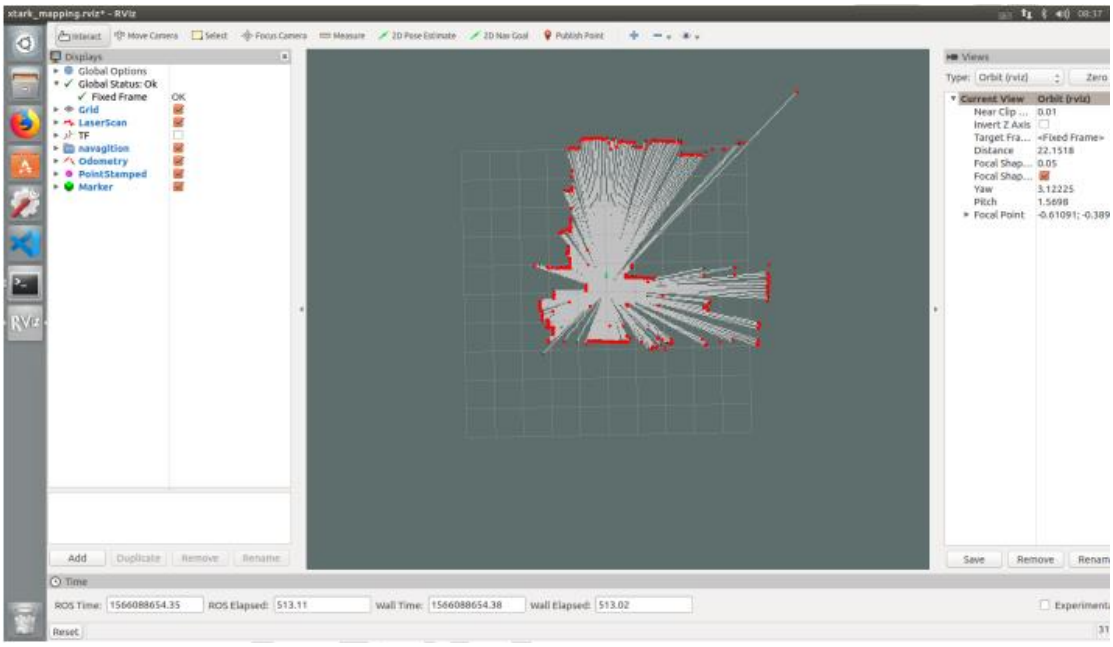
注:建图配置文件在虚拟机端。



如果出现如下提示，用户可以选择“Close without Saving”关闭不保存按钮。



加载配置文件后，可以看到雷达扫描的地图信息。



机器人建图需要控制机器人移动，可通过键盘或手柄控制机器人移动建图，下面以键

盘为例进行说明，手柄方式类似。再新建一个终端，并通过 SSH 连接到机器人，启动键

盘控制节点，通过键盘控制机器人进行移动建图。

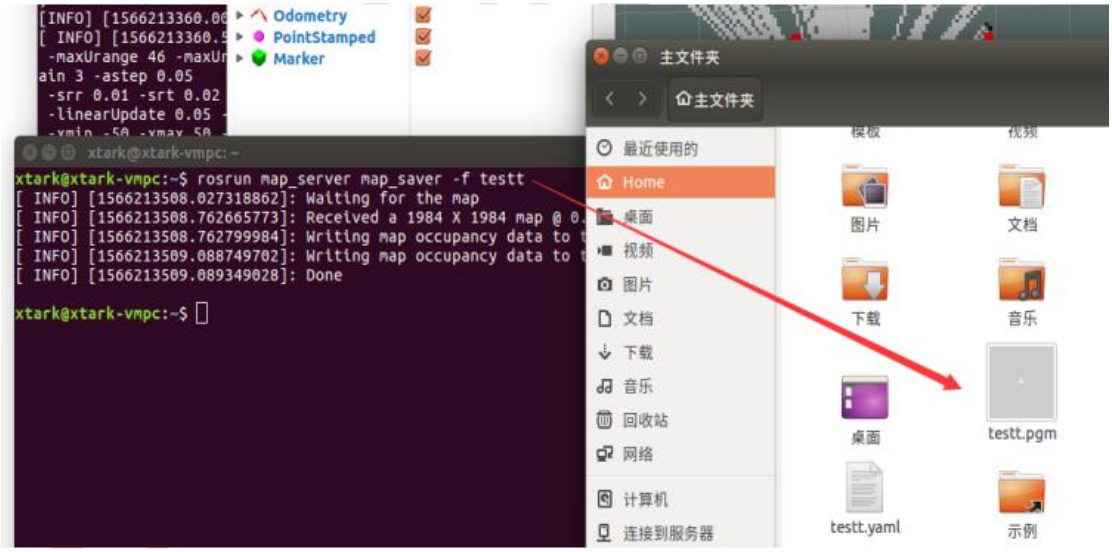
```
xtark@xtark-robot:~$ roslaunch xtark_ctl xtark_keyboard.launch
```

地图建完后，可以保存地图到机器人或开发主机，以备机器人导航时使用，保存方法如下。

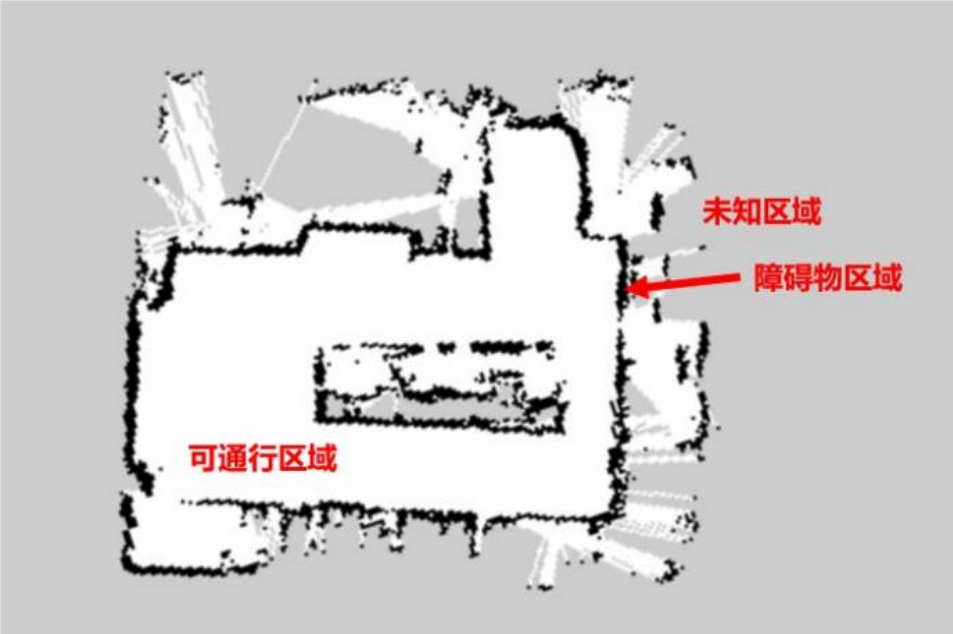
新建终端，在虚拟机端运行保存命令，可自定义地图文件名称。

```
xtark@xtark-vmpc:~$ rosrun map_server map_saver -f  自定义地图文件名称
```

虚拟机保存后的文件在主文件夹 home 中即可查看。



地图可双击查看，地图中白色为可通行区域，黑色为障碍物区域，灰色为未知区域。



2、 自主导航与避障操作

ROS 机器人支持自主导航避障，其中全向移动平台支持全向移动导航模式与差速移动导航模式，全向机器人默认为全向导航模式。启动自主导航步骤如下。

首先通过 SSH 命令连接到机器人，导航地图文件路径“~/ros_ws/src/xtark_nav/maps”，查看用于导航的地图文件。

```
xtark@xtark-robot:~/ros_ws/src/xtark_nav/maps $ ls
xtark_test_karto.pgm  xtark_test_karto.yaml  xtark_test.pgm  xtark_test.yaml
```

再新建一个终端，SSH 连接到机器人，运行“xtark_nav.launch”导航节点，map_file 参数为用于导航的地图文件(绝对路径)。

```
xtark@xtark-robot:~$ roslaunch xtark_nav xtark_nav.launch
map_file:=/home/xtark/ros_ws/src/xtark_nav/maps/地图文件.yaml
```

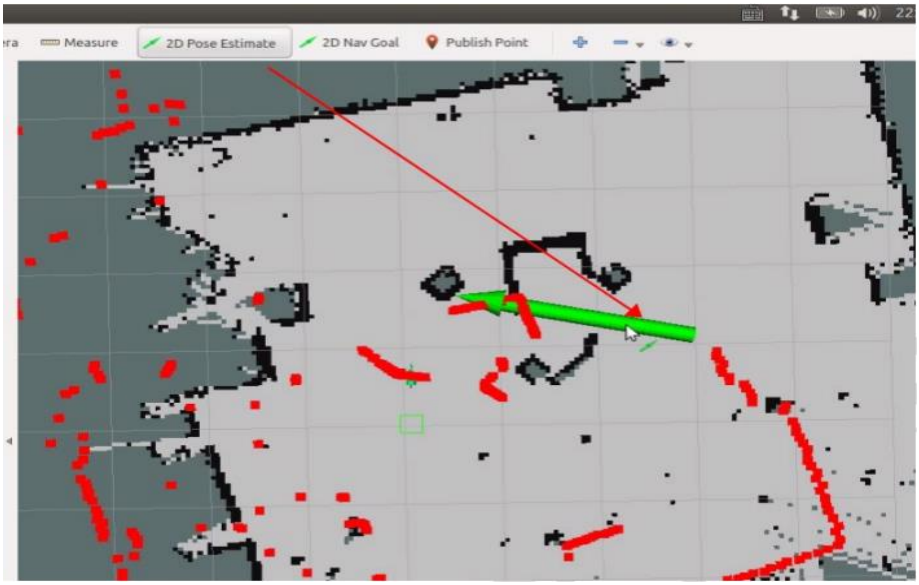
此时节点正在启动，终端会输出启动信息，等待出现下图启动完成界面。

```
 m/pix
[ INFO] [1574078384.852854158]: Received a 1984 X 1984 map at 0.050000 m/pix
[ INFO] [1574078384.866184323]: Using plugin "obstacle_layer"
[ INFO] [1574078384.875949890]:     Subscribed to Topics: laser_scan_sensor
[ INFO] [1574078384.992231481]: Using plugin "inflation_layer"
[ INFO] [1574078386.364348340]: Created local_planner teb_local_planner/TebLocal
PlannerROS
[ INFO] [1574078386.657094496]: No robot footprint model specified for trajector
y optimization. Using point-shaped model.
[ INFO] [1574078386.662339801]: Parallel planning in distinctive topologies disa
bled.
[ INFO] [1574078386.662734293]: No costmap conversion plugin specified. All occu
pied costmap cells are treaten as point obstacles.
[ INFO] [1574078387.607372138]: Recovery behavior will clear layer obstacle_laye
r
[ INFO] [1574078387.766937589]: odom received!
```

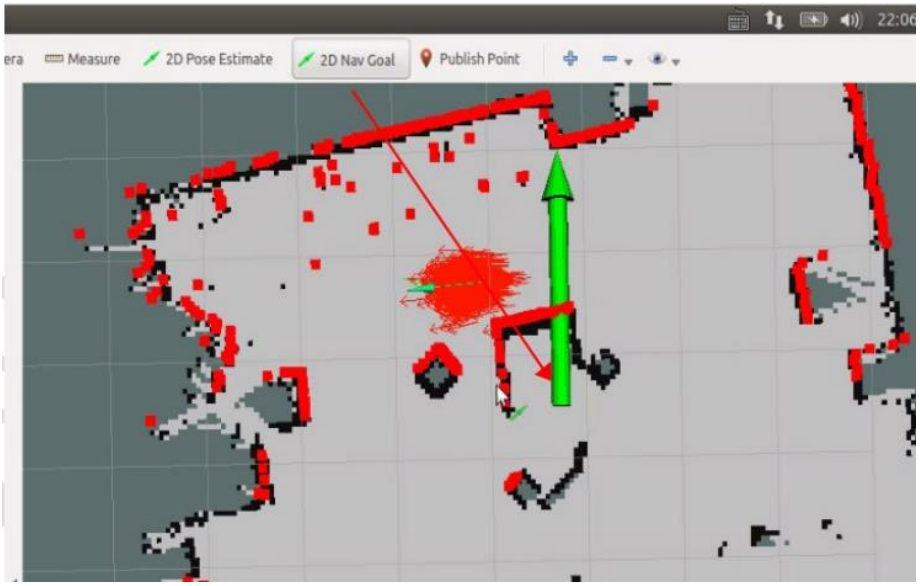
再新建一个终端，在虚拟机端运行 RVIZ 可视化工具，并在 RVIZ 加载导航配置文件"xtark_nav.rviz”。

```
xtark@xtark-vmpc:~$ rosrun rviz rviz
```

标定机器人当前位置和朝向，点击“2D Pose Estimate”按钮，将机器人的大体位置和朝向在地图中标出，箭头的尾部为 ROS 机器人的位置，箭头的方向为 ROS 机器人的朝向。



利用“2D Nav Goal”按钮发布一个目标位置，ROS 机器人即可自动导航与避障到达目标位置。



特别说明，如果用户经常在一个固定环境下使用无需频繁更换地图，也可以修改 launch 文件中的默认地图文件，实现不带参数启动导航。

```
xtark@xtark-robot:~$ roslaunch xtark_nav xtark_nav.launch
```

新建终端，通过 SSH 命令连接到机器人，进入 xtark_nav 功能包的 launch 文件夹，利用 vi 编辑器修改“xtark_nav.launch”导航 launch 文件的地图路径。

```
xtark@xtark-robot:~/ros_ws/src/xtark_nav/launch $ ls
include                          xtark_mapping_hector.launch  xtark_slam.launch
xtark_mapping_frontier.launch    xtark_mapping_karto.launch
xtark_mapping_gmapping.launch    xtark_nav.launch
xtark@xtark-robot:~/ros_ws/src/xtark_nav/launch $ vi xtark_nav.launch
```

找到导航地图文件路径和名称，默认地图存放在"/ros_ws/src/xtark_nav/maps”路径下。

```
xtark@xtark-robot:~/ros_ws/src/xtark_nav/maps $ ls
xtark_test_karto.pgm  xtark_test_karto.yaml  xtark_test.pgm  xtark_test.yaml
```

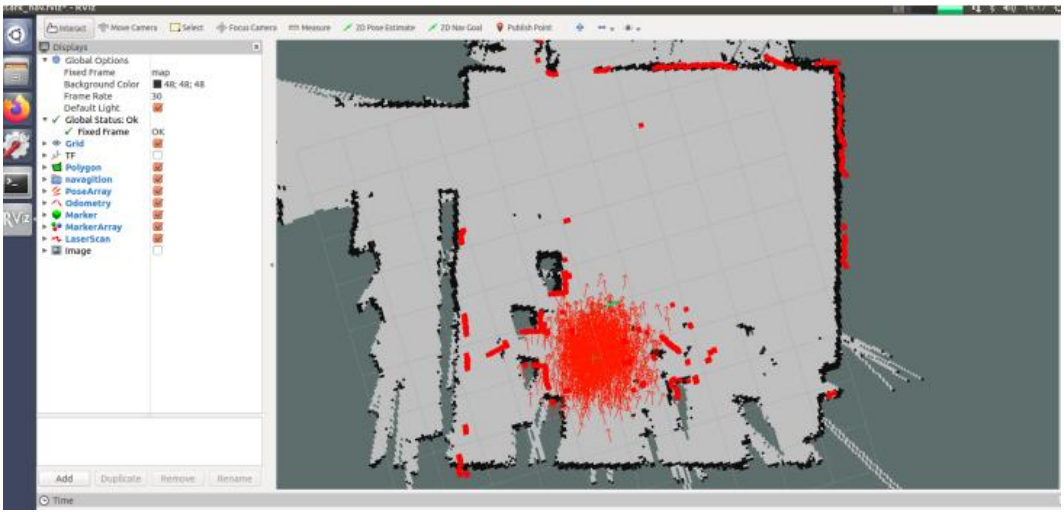
将文件名修改为需要导航的地图名称，修改完成后保存后退出。

```
<launch>
        <include file="$(find xtark_driver)/launch/xtark_bringup.launch" />

        <!-- Bring up Camera -->
        <include file="$(find usb_cam)/launch/usb_cam-map.launch"/>


        <arg name="map_file" default="$(find xtark_nav)/maps/xtark_test.yaml"/>
        <node name="map_server" pkg="map_server" type="map_server" args="$(arg map_file)" />

        <arg name="initial_pose_x" default="0.0"/>
        <arg name="initial_pose_y" default="0.0"/>
        <arg name="initial_pose_a" default="0.0"/>
        <include file="$(find xtark_nav)/launch/include/amcl_omni.launch">
                <arg name="initial_pose_x" value="$(arg initial_pose_x)"/>
                <arg name="initial_pose_y" value="$(arg initial_pose_y)"/>
                <arg name="initial_pose_a" value="$(arg initial_pose_a)"/>
        </include>
```

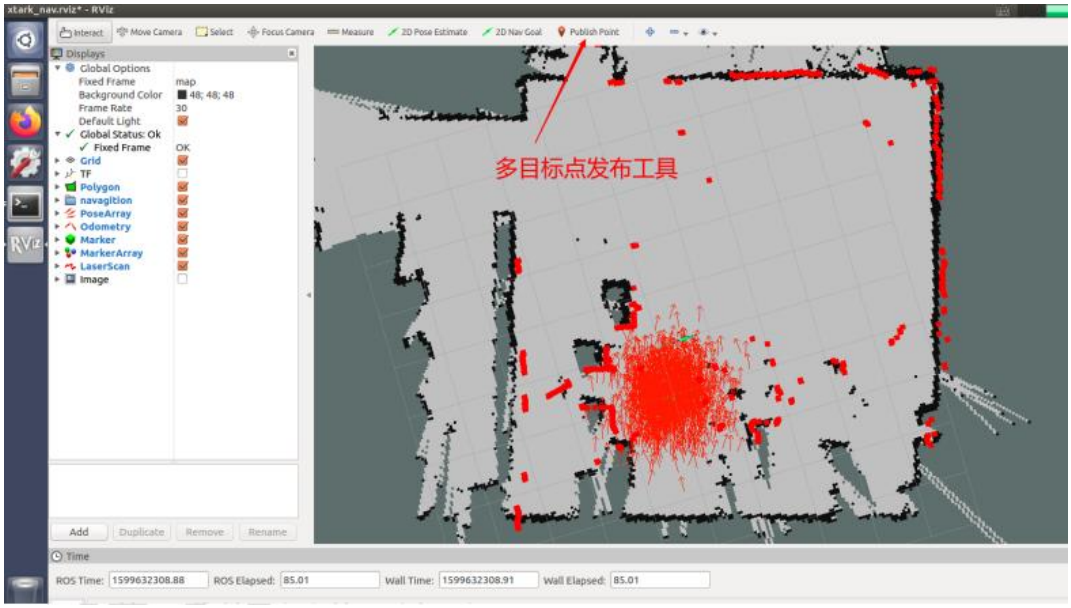
3、 多点连续导航

ROS 机器人在前文所提到的单点导航的基础上，还可实现多点连续导航，操作方法如下：

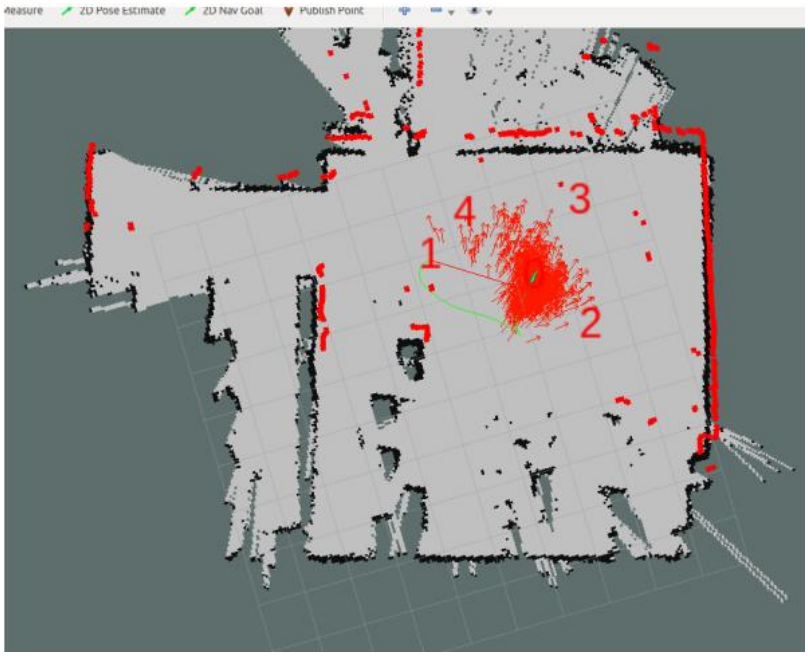
确保机器人已按照上一节所述步骤，启动了 xtark_nav.launch 并已经标定好机器人位置：



此时，利用 rviz 中的 Publish Point 工具，即可发布多个目标点，机器人将会依次前往设置的多个目标点:



点击 Publish Point 后，再点击地图上的期望目标点位置:



机器人即可按照目标点顺序，前往各个目标点。

源码路径:~/ros_ws/src/xtark_nav/scripts/　(机器人端)

4、 实验改进

修改实验内容 1 中的算法参数，观察 SLAM 激光雷达建图的效果。

四、 实验报告要求

1、 根据实验内容和步骤完成和记录实验过程与结果。
2、 修改算法参数，记录 SLAM 激光雷达建图的结果。
3、 观察 ROS 机器人的自主导航与避障，记录实验结果。